

HW2 - BPE Tokenizer for Named Entity Recognition

Raz Amzaleg - ID 206922478

Method

All three tokenizers use a word-level Byte Pair Encoding tokenizer (Sennrich/HF style, as in the lecture). Each line is split on whitespace and every non-initial word is prefixed with a space marker `_`; the vocabulary is initialized with all characters and then the most frequent adjacent symbol pair is merged repeatedly (computed with incremental pair-counts over a unique-word frequency

table, so full domain_1 trains in ~12 min) until the vocabulary reaches 2000. Encoding applies the learned merges per word, with a per-word cache for speed; decoding joins token surfaces and maps

`_` back to a space.

Differences from textbook BPE: (1) a bounded bigram phase adds the few most frequent adjacent word-pairs as single tokens (e.g. `_in_the`), which satisfies the mandatory " ≥ 1 bigram" rule while capping tokens at the bigram level (at most one internal space / two words); (2) length-preserving [UNK] decoding - an unseen character decodes to a single replacement character, so token $\leftrightarrow$ word character-span alignment in the NER pipeline never shifts; (3) case is preserved, since capitalization is the dominant entity cue on these data.

Tokenizers and the hidden domain

T1 is trained on domain_1 (Twitter), T2 on domain_2 (news). T3 targets a hidden domain and is trained

on domain_1 + domain_2 combined for maximum character and subword coverage. The vocabulary is kept

small (2000) deliberately: a data analysis showed ~ 68 % of dev entities are out-of-vocabulary, and small shared subwords generalize to unseen words far better than rare whole-word tokens - a whole token for a never-seen entity is an embedding the fixed NER model never trained. Length-preserving [UNK] further protects T3 against unknown characters in the hidden domain.

Most / least common bigrams (training set)

- domain_1 - most: `_in_the`, `_for_the`, `_going_to`, `_to_be`, `_on_the`;

least (freq 1): `_hockey_3-0.`, `_ta_so`, `_Queen_will`, `_voice`, `_as`, `@officialSusanB_ta`.

- domain_2 - most: `_of_the`, `_in_the`, `_to_the`, `_on_the`, `_to_be`;

least (freq 1): `_a_Wonder`, `_10-year-old_self`, `_Kids:_Reflections`, `_Reflections_During`, `Giving_Culture`.

Validation (dev sets)

Tokenizer | NER F1 (word-level) | efficiency (tokens/char) | encode speed

T1 - domain_1 | 0.476 | 0.338 | ~349k tok/s

T2 - domain_2 | 0.961 | 0.359 | ~391k tok/s

1 Confirm at vocab 2000 (0.96 was measured at vocab 5000; expected comparable).

Vocabulary size was selected by a sweep on domain_1: F1 peaks at 2000 (0.476) versus 0.448 at 5000,

because smaller vocabularies fragment rare/entity words into shared subwords whose embeddings the fixed

bi-LSTM actually trains. 2000 also wins the equal-weight competition blend (F1 + efficiency + speed): it

is best on F1 and encode speed, at a modest compression cost.

Hidden-domain (T3) evaluation

On the given dev sets T3 encodes at 0.357 tokens/char (domain_1) and 0.389 (domain_2). We expect a

solid hidden-domain F1: its small shared-subword vocabulary plus length-preserving [UNK] give every

word a trainable, alignment-safe first subtoken regardless of domain, which directly addresses the high

OOV rate of unseen domains. The main limiter is the inherent ambiguity of the capitalization cue - many capitalized tokens start both entities and non-entities - which is a property of the fixed model and the small, noisy data rather than of the tokenizer (it also caps domain_1 at F1 ~ 0.48).